React 핵심 이론 및 실습 코드

> 대상: React 입문 개발자 · 환경: React 함수 컴포넌트, Hooks, Vite
> 범위: 조건부 렌더링, 리스트와 키, 폼, State 끌어올리기, 합성 vs 상속, Context API, 스타일링

전체 학습 구성

번호	주제	핵심 개념	실습
1	조건부 렌더링	`if`, 삼항 연산자, `&&`	로그인 UI
2	리스트와 키	`map`, `filter`, `key`	Todo 목록
3	폼	제어 컴포넌트, 이벤트	회원가입 폼
4	State 끌어올리기	부모 상태, Props, 콜백	입력창 동기화
5	합성 vs 상속	`children`, 재사용	공통 카드
6	Context API	`createContext`, `useContext`	다크 모드
7	스타일링	CSS, Inline Style, CSS Modules	상품 카드

실습 환경

```
npm create vite@latest react-basic -- --template react
cd react-basic
npm install
npm run dev
```

별도 표시가 없는 JSX 예제는 `src/App.jsx`에 저장합니다. 각 실습은 독립적인 예제이므로 `App.jsx` 코드를 교체하며 실행합니다.

---

1. 조건부 렌더링 (Conditional Rendering)

1.1 핵심 이론

조건에 따라 서로 다른 JSX를 출력하는 기능입니다. `if`는 복잡한 분기, 삼항 연산자 `? :`는 두 화면 중 하나를 선택할 때, `&&`는 조건이 참일 때만 표시할 때 적합합니다. 렌더링에서 `null`을 반환하면 출력하지 않습니다.

방식	예시	용도
`if`	`if (loading) return <p>로딩 중</p>`	조기 반환
`? :`	`{login ? <Home /> : <Login />}`	양자택일
`&&`	`{login && <Profile />}`	조건부 표시

1.2 실습: 로그인 상태 변경

```
import { useState } from "react";

export default function App() {
  const [isLogin, setIsLogin] = useState(false);

  return (
    <div>
      <h2>조건부 렌더링 실습</h2>
      <h3>{isLogin ? "환영합니다!" : "로그인이 필요합니다."}</h3>
      {isLogin && <p>마이페이지 이용 가능</p>}
      <button onClick={() => setIsLogin(prev => !prev)}>
        {isLogin ? "로그아웃" : "로그인"}
      </button>
    </div>
  );
}
```

학습 포인트: `setIsLogin`이 상태를 바꾸면 컴포넌트가 다시 렌더링됩니다. `count && <p>...</p>`에서 `count`가 `0`이라면 화면에 숫자 0이 출력될 수 있으므로, `count > 0 && ...`처럼 명시적 불리언 조건을 사용하는 것이 안전합니다.

---

2. 리스트와 키 (Lists and Keys)

2.1 핵심 이론

배열의 각 항목을 `map()`으로 JSX에 변환합니다. `filter()`는 항목을 선택하거나 제거할 때 유용합니다. `key`는 React가 리스트 항목의 정체성을 파악하는 데 사용하는 값입니다.

```
const todos = [{ id: 1, title: "React 공부" }];
const items = todos.map(todo => <li key={todo.id}>{todo.title}</li>);
```

가능하면 데이터에 이미 존재하는 **안정적이고 고유한 ID**를 `key`로 사용합니다. 항목 삭제·삽입·재정렬이 있는 목록에서 배열 인덱스를 키로 쓰면 UI 상태가 잘못 연결될 수 있습니다. `key`는 자식 컴포넌트의 일반 Props가 아니므로 ID가 필요하면 별도의 `id` Props로 전달해야 합니다.

2.2 실습: Todo 목록 추가, 완료, 삭제

```
import { useState } from "react";

export default function App() {
  const [todos, setTodos] = useState([
    { id: "1", title: "React 공부", done: false },
    { id: "2", title: "운동하기", done: true },
  ]);
  const [text, setText] = useState("");

  function addTodo() {
    if (!text.trim()) return;
    setTodos(prev => [
      ...prev,
      { id: crypto.randomUUID(), title: text.trim(), done: false },
    ]);
    setText("");
  }

  function deleteTodo(id) {
    setTodos(prev => prev.filter(todo => todo.id !== id));
  }

  function toggleTodo(id) {
    setTodos(prev => prev.map(todo =>
      todo.id === id ? { ...todo, done: !todo.done } : todo
    ));
  }

  return (
    <div>
      <h2>Todo 리스트</h2>
      <input value={text} onChange={e => setText(e.target.value)}
             placeholder="할 일 입력" />
      <button onClick={addTodo}>추가</button>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <input type="checkbox" checked={todo.done}
                   onChange={() => toggleTodo(todo.id)} />
            <span>{todo.title}</span>
            <button onClick={() => deleteTodo(todo.id)}>삭제</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

학습 포인트: 상태 배열을 직접 `push`, `splice` 등으로 변경하는 대신 새로운 배열을 만들어 `setTodos`에 전달합니다.

---

3. 폼 (Forms)

3.1 핵심 이론

React에서 폼의 입력값은 상태로 관리하는 **제어 컴포넌트**와, DOM에서 필요한 시점에 값을 읽는 **비제어 컴포넌트** 방식으로 처리할 수 있습니다.

구분	제어 컴포넌트	비제어 컴포넌트
값의 주된 관리	React State	DOM
값 접근	`useState`	`ref`, `FormData`
실시간 검증	용이	별도 처리 필요
사용 예시	회원가입, 실시간 검색	단순 폼, 파일 입력

제어 컴포넌트의 흐름: **입력 → `onChange` → `setState` → 재렌더링 → `value` 갱신**.

3.2 실습: 회원가입 폼

```
import { useState } from "react";

export default function App() {
  const [form, setForm] = useState({
    name: "", email: "", password: "",
  });
  const [message, setMessage] = useState("");

  function handleChange(e) {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    if (!form.name.trim() || !form.email || !form.password) {
      setMessage("모든 항목을 입력하세요.");
      return;
    }
    setMessage(`${form.name}님, 가입 정보가 확인되었습니다.`);
    // 실제 서비스에서는 백엔드 API 호출 및 서버 검증 필요
  }

  return (
    <form onSubmit={handleSubmit}>
      <h2>회원가입</h2>
      <input name="name" placeholder="이름" value={form.name}
             onChange={handleChange} />
      <input name="email" type="email" placeholder="이메일"
             value={form.email} onChange={handleChange} />
      <input name="password" type="password" placeholder="비밀번호"
             value={form.password} onChange={handleChange} />
      <button type="submit">가입하기</button>
      <p>{message}</p>
    </form>
  );
}
```

학습 포인트: `e.preventDefault()`로 기본 제출 시 페이지 이동을 막습니다. `[name]: value`는 변경된 필드만 갱신하는 계산된 속성 이름 문법입니다. 프론트엔드 검증만으로 보안을 보장할 수 없으므로 서버에서 반드시 다시 검증합니다.

---

4. State 끌어올리기 (Lifting State Up)

4.1 핵심 이론

형제 컴포넌트 간에 같은 데이터를 공유해야 한다면, 해당 컴포넌트들의 **가장 가까운 공통 부모**로 상태를 옮깁니다. 부모에서 자식으로 값과 콜백을 Props로 내려보냅니다.

```
            부모 App: text State
                 /           \
          value /             \ value
               v               v
         입력창 A           입력창 B
               \               /
                \ onChange    / onChange
                 > 부모 상태 변경
```

4.2 실습: 두 입력창 동기화

```
import { useState } from "react";

function TextInput({ label, value, onTextChange }) {
  return (
    <div>
      <label>{label}: </label>
      <input value={value}
             onChange={e => onTextChange(e.target.value)} />
    </div>
  );
}

export default function App() {
  const [text, setText] = useState("");

  return (
    <div>
      <h2>State 끌어올리기</h2>
      <TextInput label="입력창 A" value={text}
                 onTextChange={setText} />
      <TextInput label="입력창 B" value={text}
                 onTextChange={setText} />
      <p>공유 데이터: {text}</p>
    </div>
  );
}
```

학습 포인트: 두 입력창은 독립적인 상태를 가지지 않습니다. 부모의 단일 상태를 읽고, 이벤트 발생 시 부모의 콜백을 호출합니다. 무조건 `App` 최상단으로 올리기보다 필요한 범위까지만 공유하는 것이 좋습니다.

---

5. 합성 vs 상속 (Composition vs Inheritance)

5.1 핵심 이론

React는 UI 재사용을 위해 상속보다 **합성**을 권장합니다. 합성은 여러 컴포넌트를 조립하는 방식이며, `children` Props를 통해 래퍼 컴포넌트 안에 임의의 내용을 삽입할 수 있습니다.

구분	합성	상속
재사용 방식	`children`, Props 조합	`extends`
주요 장점	유연한 구조, 낮은 결합도	공통 클래스 동작 재사용
React UI 권장	권장	일반적인 UI 재사용에서는 비권장

5.2 실습: 재사용 가능한 카드

```
function Card({ title, children }) {
  const style = {
    border: "1px solid #ccc",
    borderRadius: "10px",
    padding: "20px",
    marginBottom: "15px",
  };

  return (
    <div style={style}>
      <h3>{title}</h3>
      {children}
    </div>
  );
}

function UserInfo() {
  return (
    <Card title="사용자 정보">
      <p>이름: 홍길동</p>
      <p>직업: 개발자</p>
    </Card>
  );
}

function ProductInfo() {
  return (
    <Card title="상품 정보">
      <p>상품명: 노트북</p>
      <p>가격: 1,200,000원</p>
      <button>구매하기</button>
    </Card>
  );
}

export default function App() {
  return (
    <div>
      <h2>컴포넌트 합성 실습</h2>
      <UserInfo />
      <ProductInfo />
    </div>
  );
}
```

상속의 형태는 `class ProductCard extends BaseCard`와 같지만, React에서는 일반적으로 `<Card>상품 UI</Card>`처럼 컴포넌트를 조합합니다.

학습 포인트: `children`은 컴포넌트 태그 사이에 배치된 JSX입니다. 공통 레이아웃·카드·모달·대화상자 등에 활용할 수 있습니다.

---

6. Context 및 Context API

6.1 핵심 이론

Context API는 여러 단계의 중간 컴포넌트에서 Props를 매번 전달하는 **Props Drilling**을 줄입니다. 대표 사용 사례는 테마, 로그인 사용자, 언어, 전역 환경 설정입니다.

- `createContext(defaultValue)`: Context 객체 생성
- `Context.Provider`: 하위 트리에 값 제공 (React 18 포함 호환 형식)
- `useContext(Context)`: 가장 가까운 상위 Provider의 값 읽기
- `useState`: Context로 제공할 동적 상태 관리에 사용할 수 있음

React 19에서는 `<ThemeContext value={...}>` 형태도 가능하지만, 아래는 React 18과 19 모두 이해하기 쉬운 `.Provider` 형식을 사용합니다.

6.2 실습: Context를 이용한 다크 모드

프로젝트 구조:

```
src/
├── App.jsx
├── ThemeContext.jsx
├── Header.jsx
└── main.jsx
```

src/ThemeContext.jsx

```
import { createContext } from "react";

export const ThemeContext = createContext(null);
```

src/Header.jsx

```
import { useContext } from "react";
import { ThemeContext } from "./ThemeContext";

export default function Header() {
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <header>
      <h2>현재 테마: {theme}</h2>
      <button onClick={toggleTheme}>테마 변경</button>
    </header>
  );
}
```

src/App.jsx

```
import { useState } from "react";
import { ThemeContext } from "./ThemeContext";
import Header from "./Header";

export default function App() {
  const [theme, setTheme] = useState("light");

  function toggleTheme() {
    setTheme(prev => prev === "light" ? "dark" : "light");
  }

  const isDark = theme === "dark";

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      <div style={{
        backgroundColor: isDark ? "#222" : "#fff",
        color: isDark ? "#fff" : "#222",
        minHeight: "100vh", padding: "20px",
      }}>
        <Header />
        <p>Context API 실습 예제입니다.</p>
      </div>
    </ThemeContext.Provider>
  );
}
```

학습 포인트: Context는 데이터를 하위 트리에 전달하는 메커니즘이지, 그 자체가 독립적인 상태 저장소는 아닙니다. Provider 값이 바뀌면 해당 Context를 구독하는 컴포넌트가 재렌더링될 수 있습니다. 단순한 부모-자식 관계에서는 Props가 더 직관적입니다.

---

7. 스타일링 (Styling)

7.1 주요 방식 비교

방식	특징	활용
일반 CSS	`.css` 파일에서 클래스 선언	기본 웹 UI

방식	특징	활용
Inline Style	JSX `style`에 JavaScript 객체 전달	동적 스타일
CSS Modules	`.module.css`로 클래스 범위 분리	컴포넌트 스타일
Tailwind CSS	유틸리티 클래스 조합	빠른 UI 구현
CSS-in-JS	JavaScript 기반 스타일 정의	일부 프로젝트의 동적 테마

7.2 일반 CSS 실습

src/App.jsx

```
import "./App.css";

export default function App() {
  return (
    <div className="card">
      <h2>React 스타일링</h2>
      <p>일반 CSS 적용</p>
    </div>
  );
}
```

src/App.css

```
.card {
  background-color: #f1f5f9;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  padding: 20px;
}
.card h2 { color: #2563eb; }
```

7.3 Inline Style 실습

```
export default function App() {
  const isActive = true;
  return (
    <button style={{
      backgroundColor: isActive ? "blue" : "gray",
      color: "white",
      padding: "10px 20px",
      borderRadius: "8px",
    }}>
     상태 버튼
    </button>
  );
}
```

HTML의 `class` 대신 JSX에서는 `className`, CSS의 `background-color` 대신 스타일 객체에서는 `backgroundColor`를 사용합니다. 숫자형 스타일 값은 속성별로 단위 처리 규칙이 다릅니다.

7.4 CSS Modules 실습

src/ProductCard.module.css

```
.card {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 20px;
  background-color: white;
}
.title { color: #2563eb; }
.price { font-weight: bold; color: #dc2626; }
```

src/ProductCard.jsx

```
import styles from "./ProductCard.module.css";

export default function ProductCard() {
  return (
    <div className={styles.card}>
      <h3 className={styles.title}>노트북</h3>
      <p className={styles.price}>1,200,000원</p>
    </div>
  );
}
```

Vite에서 `.module.css`를 지원하므로 기본적인 CSS Modules 사용에 추가 라이브러리가 필요하지 않습니다.

7.5 실습: 상태에 따라 선택 스타일 변경

src/App.jsx

```
import { useState } from "react";
import "./App.css";

export default function App() {
  const [selected, setSelected] = useState(false);

  return (
    <div className="container">
      <div className={`product-card ${selected ? "selected" : ""}`}>
        <div className="product-image">💻</div>
        <h2>React 실습용 노트북</h2>
        <p className="price">1,200,000원</p>
        <button onClick={() => setSelected(prev => !prev)}>
          {selected ? "선택 해제" : "상품 선택"}
        </button>
      </div>
    </div>
  );
}
```

src/App.css

```
.container {
  display: flex;
  justify-content: center;
  padding: 40px;
}
.product-card {
  width: 280px;
  padding: 24px;
  border: 2px solid #ddd;
  border-radius: 14px;
  text-align: center;
  transition: 0.2s;
}
.product-card.selected {
  border-color: #2563eb;
  background-color: #eff6ff;
}
.product-image { font-size: 60px; }
.price { color: #dc2626; font-weight: bold; }
button {
  padding: 10px 20px;
  border: none;
  border-radius: 8px;
  background-color: #2563eb;
  color: white;
  cursor: pointer;
}
```

학습 포인트: `className={selected ? "active" : "inactive"}`처럼 State에 따라 스타일을 바꾸는 패턴이 중요합니다.

---

8. 종합 실습: React Todo 관리 앱 설계

일곱 가지 학습 개념을 하나의 앱에서 연결합니다.

주제	종합 프로젝트 적용
조건부 렌더링	할 일 유무 및 완료 안내
리스트와 키	Todo 목록과 고유 ID
폼	Todo 입력창
State 끌어올리기	Form과 List가 부모의 Todo 상태 공유
합성	공통 `Card`, `Layout`
Context API	테마 전환
스타일링	CSS 및 CSS Modules

추천 폴더 구조:

```
src/
├── components/
│   ├── TodoForm.jsx
│   ├── TodoList.jsx
│   ├── TodoItem.jsx
│   ├── Card.jsx
│   └── Header.jsx
├── contexts/
│   └── ThemeContext.jsx
├── styles/
│   └── App.css
├── App.jsx
└── main.jsx
```

상태 흐름: `App`가 Todo State를 관리하고 `TodoForm`에 `onAddTodo`, `TodoList`에 `todos` 및 수정·삭제 콜백을 전달합니다. `TodoList`는 `TodoItem` 목록을 렌더링합니다. `ThemeContext.Provider`는 테마 값을 하위 컴포넌트에 제공할 수 있습니다.

권장 학습 시간

학습 항목	이론	실습
조건부 렌더링	20분	30분
리스트와 키	30분	40분
폼	40분	60분
State 끌어올리기	30분	40분
합성 vs 상속	20분	30분
Context API	40분	60분
스타일링	40분	60분
합계	**220분**	**320분**

총 9시간이며 실습·복습까지 고려하면 약 1.5~2일 과정으로 편성할 수 있습니다.

마무리

React의 핵심은 **State 변경 → 컴포넌트 재렌더링 → UI 갱신**입니다. 조건부 렌더링, 리스트, 폼은 이를 실제 화면에 적용하는 방법이며, State 끌어올리기와 Context는 데이터를 공유하는 설계 방법입니다. 합성과 스타일링은 UI의 재사용성과 표현을 담당합니다.

공식 참고 문서

- 조건부 렌더링
- 리스트 렌더링
- State 공유
- Context로 데이터 전달
- 입력 폼